

0x01

指针和结构体

C 语言内存布局 · 逆向工程基础篇

学习要点整理 · 含前置知识清单与大量实例

本讲义最终要彻底讲清的两行代码

```
*( _BYTE *) ( *( _QWORD *) (a1 + 8208) + 4120LL )  
(const char *) (a1 + 4112)
```

这两行看起来吓人，其实只用到三个概念：字节、指针、结构体。本讲义把它们拆成 7 个部分（第 0 章前置清单 + 第 1~6

章正文），从二进制与十六进制开始，一路推到能独立读懂上面这两行，并且能反推出它对应的 C 结构体定义。

怎么用这份讲义

先看第 0 章的前置清单，能打钩的跳过；第 1~4 章是地基，必须动手写代码验证；第 5 章是全篇的落点，把逆向代码逐层剥开；第 6 章的自测题做完再看答案。每一节都配了可编译运行的例子，建议全部实际跑一遍。

适用起点：学过 C 的基本语法（变量、if、for、函数），但没系统学过指针与内存。

目标终点：能看懂 IDA / Ghidra 反编译出的结构体访问与指针解引用代码。

第 0 章 前置知识清单

这一章不是“可看可不看”的引言，而是一张检查表。指针之所以难，绝大多数时候不是指针本身难，而是它依赖的下层知识有洞：不知道一个字节里放什么、不知道地址从哪来、不知道类型决定读几个字节。请逐条自评，做不到的条目记下编号，先去补。

0.1 必需项（缺一项都会卡住）

#	能力项	自检标准：能独立做到这些吗
1	二进制 / 十六进制 / 十进制互转	看到 0x2010 能立刻反应出是十进制 8208；看到 4120 能写出 0x1018
2	字节 (byte) 与位 (bit)	知道 1 字节 = 8 位 = 2 个十六进制数字；0xFF 是多少十进制
3	内存是“带地址的字节数组”	能把一块内存想象成一排编号格子，每格恰好 1 字节（8 位）
4	地址与地址宽度	知道 x86 用 4 字节地址、x64 用 8 字节地址；知道“地址就是无符号整数”

5	大小端（字节序）	能说出 0x11223344 在内存里按小端是 44 33 22 11
6	C 的基本类型与 sizeof	能用 sizeof 打印 char/short/int/long long/指针 的大小，并解释结果
7	变量的地址与取地址符 &	能写出 int a=1; printf("%p", (void*)&a); 并解释输出的含义
8	C 基本语法与编译运行	能写 main 函数、用 printf、用 gcc/clang/MSVC 编译并运行
9	数组的基础	知道 int a[5] 的 5 个元素在内存里是连续排列的
10	调试器最基础操作	能在 gdb 或 x64dbg 里给变量下断点、查看内存窗口

表 0-1 前置知识必需项

0.2 加分层（有则更快）

- Hex 编辑器：十六进制编辑器：能打开一个二进制文件并定位任意偏移。
- 汇编入门：能看懂 mov / lea / add / call 这几条指令，知道寄存器大概是干什么的。
- PE 结构：知道 EXE / DLL / PE 段（.text / .data / .rdata）大致区别。
- 栈与调用约定：知道栈向下增长、函数调用会有栈帧。
- 字符编码：知道 ASCII 码：字符 A 是 0x41，数字 0 是 0x30，字符串以 0 结尾。

0.3 第 0 章的几个最小验证实验

下面三个程序很短，但它们是后面所有内容的显微镜。请务必跑一遍，并且把输出和你心里的预期逐个对照。

实验 1：看清楚一个整数的四个字节

```
exp1.c —— 观察小端序
#include <stdio.h>

int main(void)
{
    int a = 0x11223344;          /* 四个字节: 11 22 33 44 */
    unsigned char *p = (unsigned char *)&a; /* 把 int* 当 1 字节为单位看 */

    for (int i = 0; i < 4; i++)
        printf("第 %d 个字节(偏移+%d) = 0x%02X\n", i, i, p[i]);

    return 0;
}
```

典型输出（x86/x64 小端）：

```
第 0 个字节(偏移+0) = 0x44
第 1 个字节(偏移+1) = 0x33
第 2 个字节(偏移+2) = 0x22
第 3 个字节(偏移+3) = 0x11
```

这个实验说明了什么

一是"变量在内存里就是连续的几个字节"; 二是"低地址放低位字节" (小端); 三是"同一个地址, 用不同类型去看, 看到的宽度不同"。这三点是指针全部的直觉来源。

实验 2: sizeof 与地址宽度

exp2.c —— 类型尺寸表

```
#include <stdio.h>

int main(void)
{
    printf("char      %zu\n", sizeof(char));
    printf("short     %zu\n", sizeof(short));
    printf("int        %zu\n", sizeof(int));
    printf("long        %zu\n", sizeof(long));
    printf("long long   %zu\n", sizeof(long long));
    printf("float       %zu\n", sizeof(float));
    printf("double      %zu\n", sizeof(double));
    printf("void*        %zu\n", sizeof(void *));
    printf("char*        %zu\n", sizeof(char *));
    printf("int*         %zu\n", sizeof(int *));
    return 0;
}
```

在 64 位 Windows (MSVC / MinGW64) 上, long 是 4 字节; 在 64 位 Linux 上, long 是 8 字节。这是跨平台逆向时最容易踩的坑之一。

实验 3: 把地址当数字打印

exp3.c —— 地址也是数字

```
#include <stdio.h>
#include <stdint.h>

int g_value = 0x1234; /* 全局变量, 位于 .data */

int main(void)
{
    int local = 0x5678; /* 局部变量, 位于栈 */
    printf("g_value 的地址 = %p\n", (void *)&g_value);
    printf("local 的地址 = %p\n", (void *)&local);
    printf("main 的地址 = %p\n", (void *)&main);
    /* 地址本质是无符号整数, 可以转成整数来看 */
    printf("地址的数值    = 0x%lX\n",
           (unsigned long long)(uintptr_t)&g_value);
    return 0;
}
```

为什么这很重要

逆向时你看到的 $a1 + 8208$ 这种写法，本质就是"某个对象的地址 + 偏移"；能接受"地址就是数字，可以加减"，才不会被后面的指针运算绕晕。

第 1 章 内存、字节与字节序

本章把所有指针问题的最底层前提讲清楚：内存长什么样、一个值怎么被拆成字节、字节按什么顺序摆放。

1.1 内存的模型：带编号的一排格子

把内存想象成一排格子，每个格子恰好 1 字节（8 位），每个格子有一个编号，这个编号就是“地址”。地址从 0 开始，逐 1 递增。x86-32 的编号范围是 0x00000000 ~ 0xFFFFFFFF，x86-64 则是 0x0000000000000000 ~ 0xFFFFFFFFFFFFFFFF（实际可用宽度更小）。

图 1-1 内存是一排带编号的格子

地址	内容 (1 字节)
0x00002000	0x44 <- 变量 a 的第 0 个字节
0x00002001	0x33 <- 第 1 个字节
0x00002002	0x22 <- 第 2 个字节
0x00002003	0x11 <- 第 3 个字节
0x00002004	

结论：int a = 0x11223344; 占用地址 0x2000~0x2003 共 4 个格子。
a 的“地址”指的是首字节地址 0x2000。

1.2 数据宽度与十六进制表示

十六进制与二进制、字节是一一对应的，这也是为什么逆向里几乎只用十六进制。

十六进制位数	二进制位数	字节数	C 类型 (x86 常见)	IDA 类型名	取值范围示例
2 位	8	1	char / unsigned char	_BYTE	0x00 ~ 0xFF
4 位	16	2	short	_WORD	0x0000 ~ 0xFFFF
8 位	32	4	int / long	_DWORD	0x00000000 ~ 0xFFFFFFFF
16 位	64	8	long long / 指针(x64)	_QWORD	0x0 ~ 0xFFFFFFFFFFFFFFFF

表 1-1 十六进制位数与类型宽度对应关系

把这张表背下来

IDA 里的 _BYTE / _WORD / _DWORD / _QWORD 就是“读 1/2/4/8 个字节”的意思。看 _BYTE 就知道这次只取一个字节，看 _QWORD 就知道取 8 字节（通常是在取一个指针）。这是读反编译代码时最快的信息来源。

■ 越小越容易看错的地方：0x2010 与 8208

逆向代码里偏移量有时写十六进制（0x2010），有时写十进制（8208），它们是同一个数。把十六进制换算练熟，能省掉大量来回计算：

```
0x2010 = 2 * 4096 + 0 * 256 + 1 * 16 + 0 = 8192 + 16 + 0
        = 8208      <-- 与反编译代码里的 a1 + 8208 完全一致
```

反过来：4120 = 0x1018

4120 / 16 = 257 ... 8 -> 末位 8

257 / 16 = 16 ... 1 -> 次位 1

16 / 16 = 1 ... 0 -> 再一位 0

1 / 16 = 0 ... 1 -> 最高位 1

所以 4120 = 0x1018

■ 1.3 小端与大端：多字节整数怎么摆

一个 4 字节整数要放进内存，先放哪个字节？这就产生了两种约定。

- 小端（Little Endian）：低位字节放在低地址。x86 / x64 都采用小端。
- 大端（Big Endian）：高位字节放在低地址。部分 ARM、PowerPC、网络字节序采用大端。

图 1-2 同一整数在两种字节序下的内存排布

```
int a = 0x11223344;
```

小端（x86/x64）：

+0	+1	+2	+3
0x44	0x33	0x22	0x11

<-- 低位在低地址

大端：

+0	+1	+2	+3
0x11	0x22	0x33	0x44

<-- 高位在低地址

■ 为什么逆向必须在意字节序

两个非常实际的后果：

- 看内存猜值：在内存窗口里看到 44 33 22 11，要能立刻反应出“这是一个 int，值是 0x11223344”。
- 反汇编与内存对照：在反汇编里看到 mov eax, 44332211h，而内存里是 11 22 33 44，要能反应出这是同一数据的两种视角。

■ 实验 4：用 union 直观验证字节序

exp4.c —— union 是查看同一块内存不同解释的最佳工具

```
#include <stdio.h>
#include <stdint.h>

typedef union {
    uint32_t dw;      /* 4 字节整体 */
    uint8_t b[4];     /* 4 个单字节 */
} DwordBytes;

int main(void)
{
    DwordBytes u;
    u.dw = 0x11223344;

    printf("整体值 = 0x%08X\n", u.dw);
    printf("b[0]=%02X b[1]=%02X b[2]=%02X b[3]=%02X\n",
        u.b[0], u.b[1], u.b[2], u.b[3]);

    if (u.b[0] == 0x44)
        printf("=> 本机是小端 (低字节在低地址) \n");
    else
        printf("=> 本机是大端\n");
    return 0;
}
```

union 与 struct 的关键差别

struct 的成员是"依次排开", 大小约为各成员之和 (还要对齐) ; union 的成员是"全部重叠在同一处", 大小等于最大成员。逆向中 union 常出现在协议解析、寄存器定义和类型双关 (type punning) 里。

第 2 章 类型、大小与内存布局

指针的"指针类型"决定了三件完全不同的事，这一章把这三件事一次讲透。

2.1 类型的三重含义

含义	说明	例子
读几个字节	决定从这个地址取多宽的数据	char* 取 1 字节; int* 取 4 字节; _QWORD* 取 8 字节
怎么解释这些字节	同样 4 字节, 可以当 int, 也可以当 float, 也可以当 double	char* 是 1.0f, 当 int 是 1065353216
加 1 走多远	指针 +1 移动的是"一个元素"的字节数	char* +1 走 1 字节; int* +1 走 4 字节; _QWORD* +1 走 8 字节

表 2-1 指针类型决定的三件事

这是最容易被忽略的一条

很多人以为"指针就是一个地址", 于是不理解为什么还要区分 char* 和 int*。地址确实相同, 但"以该地址为起点读多少字节、以及加法放大多少倍"由类型决定。反编译代码里那些把同一个地址反复强转成不同类型的写法, 正是在切换这三种含义。

2.2 常见类型尺寸对照

类型	x86 (32位)	x64 Windows (LLP64)	x64 Linux (LP64)	说明
char / _BYTE	1	1	1	最小寻址单位
short / _WORD	2	2	2	短整型
int / _DWORD	4	4	4	最常用的整型
long	4	4	8	平台差异最大
long long	8	8	8	64 位整型
float	4	4	4	单精度
double	8	8	8	双精度
指针 / _QWORD(x64)	4	8	8	x64 下翻倍
size_t	4	8	8	跟随指针宽度

表 2-2 同一类型在不同平台的大小 (重点看 long 与指针)

平台差异带来的真实后果

一个结构体在 32 位程序里可能是 24 字节, 在 64 位程序里变成 32 字节, 成员偏移也全变。逆向时第一步必须先确认目标程序的位数 (看 PE 头或寄存器宽度), 否则算出来的偏移一定是错的。

2.3 C 语言内建类型在内存里的样子

整数：直接按字节序摆放

```
int    a = 0x11223344; /* 4 字节 */
short  s = 0x1122;     /* 2 字节 */
char    c = 0x11;      /* 1 字节 */
long long q = 0x1122334455667788LL; /* 8 字节 */
```

小端内存视图（从低地址到高地址）：

```
a : 44 33 22 11
s : 22 11
c : 11
q : 88 77 66 55 44 33 22 11
```

浮点数：IEEE 754 格式，不能“猜”

float 的 4 个字节是“符号 1 位 + 指数 8 位 + 尾数 23 位”，double 是“1 + 11 + 52”。看到 0x3F800000 就要想到 1.0f，看到 0x40490FDB 就要想到 3.1415927f。

exp5.c —— 浮点的位模式

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    float f = 1.0f;
    unsigned int bits;
    memcpy(&bits, &f, 4);
    printf("1.0f 的位模式 = 0x%08X\n", bits);      /* 0x3F800000 */

    f = 3.1415927f;
    memcpy(&bits, &f, 4);
    printf("3.1415927f = 0x%08X\n", bits);        /* 0x40490FDB */

    double d = 1.0;
    unsigned long long b64;
    memcpy(&b64, &d, 8);
    printf("1.0 = 0x%016lX\n", b64);             /* 0x3FF0000000000000 */
    return 0;
}
```

逆向中的实战价值

在内存里看到 0x3F800000 附近的数，很可能是浮点常量而不是整数；看到 0x42F60000 就该想到 123.0f。这类“识数”能力能显著加快还原速度。

指针：存的就是一个地址

```
int a = 10;
int *p = &a;
```

32 位下的内存（假设 a 在 0x0019FF2C）：

&a = 0x0019FF2C 该处 4 字节 = 0A 00 00 00 （a 的值 10）
&p = 0x0019FF20 该处 4 字节 = 2C FF 19 00 （p 的值 = a 的地址）

64 位下的内存（假设 a 在 0x000000000065FE1C）：

&a = 0x000000000065FE1C 4 字节 = 0A 00 00 00
&p = 0x000000000065FE10 8 字节 = 1C FE 65 00 00 00 00 00

结论：p 本身也是一块内存，也占空间（4 或 8 字节），
它里面装的是 a 的地址。

2.4 结构体：把多个变量打包

结构体就是"把若干个变量按声明顺序连续排列"，它是逆向中最重要的类型，因为它直接对应"对象"和"偏移表"。

结构体的声明顺序就是内存顺序

```
typedef struct _Point {
    int x;    /* 偏移 +0 */
    int y;    /* 偏移 +4 */
} Point;

typedef struct _Player {
    int id;    /* 偏移 +0 */
    int hp;    /* 偏移 +4 */
    Point pos; /* 偏移 +8 : 一个结构体嵌在另一个里 */
    char name[16]; /* 偏移 +16 */
    void *weapon; /* 偏移 +32 (x64 下因为对齐会到 +32) */
} Player;
```

各类型"尺寸"与"对齐值"的关系是理解结构体的钥匙，第 4 章会专门推导 padding

与对齐，这里只需要先建立直觉：结构体成员按声明顺序摆放，偏移量由前面成员的大小和对齐共同决定。

第 3 章 指针核心概念

这一章是重头戏。每个小节都配一个可运行例子，并且尽量给出"内存图 + 打印输出 + 逆向里长什么样"三件套。

3.1 声明、取地址、解引用

三个最基本操作

```
int a = 10;
int *p = &a; /* p 保存 a 的地址 */

printf("%d\n", a); /* 10 —— 直接访问 */
printf("%d\n", *p); /* 10 —— 解引用，取 p 指向处的值 */
*p = 20; /* 等价于 a = 20 */
```

写法	读法	含义
int *p;	p 是一个指向 int 的指针	声明：p 用来装 int 的地址
&a	取 a 的地址	结果类型是 int*
*p	解引用 p	取出 p 指向的那 4 个字节，并按 int 解释
p	指针自身的值	就是一个地址数字
&p	指针变量自己的地址	类型是 int**，即二级指针

表 3-1 指针相关写法的含义

最容易混的两个符号

声明里的 * 属于"类型" (int* 是一个类型)，表达式里的 * 是"解引用运算符"。同理 & 在表达式里是"取地址"，在声明里是"引用" (C++) 或按位与 (两操作数时)。读代码时先判断这个 * 是在类型位置还是表达式位置，能避免一大半困惑。

3.2 指针的大小与它存的内容

exp6.c —— 指针自身的尺寸与内容

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    int *p = &a;

    printf("sizeof(p)   = %zu   (指针本身占几个字节)\n", sizeof(p));
    printf("p 的值(地址) = %p\n", (void *)p);
    printf("&p 的地址   = %p   (指针变量自己也有地址)\n", (void *)&p);
    printf("*p 的值     = %d\n", *p);

    /* 把地址当整数看 */
    printf("p 作为整数   = %llu\n", (unsigned long long)(size_t)p);
    return 0;
}
```

一个反直觉但必须接受的事实

指针变量本身也占内存、也有地址。所以"二级指针"才成立：int **pp = &p; 表示 pp 指向"那个装着 a 地址的 8 字节格子"。逆向中 *(a1 + 48) 这类写法，往往就是在从结构体里取出一个指针。

3.3 指针运算：+1 到底加多少

这是初学者最常算错的地方。指针 +1 加的不是 1 个字节，而是"1 个元素"的字节数。

假设 base = 0x1000，且各类型尺寸为 char=1, short=2, int=4, long long=8:

```
char    *pc = (char *)base;    pc + 1 = 0x1001  (+1 字节)
short   *ps = (short *)base;   ps + 1 = 0x1002  (+2 字节)
int      *pi = (int *)base;     pi + 1 = 0x1004  (+4 字节)
long long *pq = (long long *)base; pq + 1 = 0x1008  (+8 字节)
```

公式：地址 = base + n * sizeof(*p)

n 可以是负数，所以 p - 1 会往低地址走一个元素

实验 7：用实际地址验证指针算术

exp7.c —— 同一个地址, 不同类型, +1 走不同距离

```
#include <stdio.h>

int main(void)
{
    int arr[4] = { 0x11111111, 0x22222222, 0x33333333, 0x44444444 };
    int *pi = arr;
    char *pc = (char *)arr;

    printf("arr      = %p\n", (void *)arr);
    printf("&arr[1] = %p\n", (void *)&arr[1]);
    printf("pi + 1 = %p 差 %lld 字节\n", (void *)(pi + 1),
        (long long)((char *)pi + 1) - (char *)pi);
    printf("pc + 1 = %p 差 %lld 字节\n", (void *)(pc + 1),
        (long long)((pc + 1) - pc));

    printf("*pi      = 0x%08X\n", *pi);
    printf("*(pi+1) = 0x%08X\n", *(pi + 1));

    /* 用 char* 窥探同一个 int 的各个字节 (小端) */
    for (int i = 0; i < 4; i++)
        printf("第 %d 个 int 的首字节 = 0x%02X\n", i, pc[i * 4]);
    return 0;
}
```

这就是为什么反编译代码里到处是强制类型转换

当逆向工具需要"按 1 字节步进"扫描一块内存时, 就会写成 (char *)p + i; 需要"按 4 字节读一个整数"时, 就写 *(DWORD *)p。强转类型不是为了改变那块内存, 而是为了改变"读取宽度与步进倍率"。

3.4 指针与数组: 名字背后的真相

在大多数表达式里, 数组名会"退化"成指向首元素的指针。这解释了下标运算和指针运算为什么完全等价。

数组与指针的等价关系

```
int a[5] = { 10, 20, 30, 40, 50 };

/* 下面四种写法完全等价 */
a[2]      == *(a + 2) == *(2 + a) == 2[a]

/* 推理过程: */
a[2] <=> *(a + 2)      下标就是"取地址再解引用"
    <=> *(2 + a)      加法可交换
    <=> 2[a]          所以 2[a] 合法 (但别这么写)

/* 地址关系 */
a      == &a[0]      数组名退化为首元素地址
a + 2   == &a[2]
*(a + 2) == a[2]
```

两个必须分清的例子

`int a[5];` `sizeof(a)` 是 20 (整个数组)，而函数参数里 `int a[]` 会退化成指针，`sizeof` 变成 4 或 8。这就是"数组作为函数参数会丢失长度信息"的根本原因，也是逆向里函数签名经常写成 `(int *a, int n)` 的由来。

实验 8：数组名退化与下标运算

exp8.c —— 数组名退化的证据

```
#include <stdio.h>

void show(int arr[], int n) /* 这里 arr 其实是指针 */
{
    printf("函数内 sizeof(arr) = %zu (已退化为指针)\n", sizeof(arr));
    for (int i = 0; i < n; i++)
        printf(" arr[%d]=%d *(arr+%d)=%d *(arr+%d)=%d\n",
            i, arr[i], i, *(arr + i), i, *(i + arr));
}

int main(void)
{
    int a[5] = { 10, 20, 30, 40, 50 };
    printf("main 里 sizeof(a) = %zu (整个数组)\n", sizeof(a));
    printf("a[2]=%d 2[a]=%d *(a+2)=%d *(2+a)=%d\n",
        a[2], 2[a], *(a + 2), *(2 + a));
    show(a, 5);
    return 0;
}
```

3.5 多级指针

多级指针听起来可怕，本质只是"指针的指针"：每一层多一次解引用，就多走一次内存跳转。

多级指针的层次

```
int a = 42;
int *p = &a; /* 一级: p -> a */
int **pp = &p; /* 二级: pp -> p -> a */

printf("%d\n", a); /* 42 */
printf("%d\n", *p); /* 42 */
printf("%d\n", **pp); /* 42 —— 解两次 */

/* 每一层的类型 */
/* a : int 值 = 42 */
/* p : int* 值 = &a */
/* pp : int** 值 = &p */
/* &pp : int*** 值 = pp 自己的地址 */
```

实验 9：二级指针的三层内存

exp9.c —— 二级指针同时演示"改指向"和"改值"

```
#include <stdio.h>

int main(void)
{
    int    a   = 42;
    int    *p   = &a;
    int    **pp = &p;

    printf("&a   = %p    a   = %d\n", (void *)&a, a);
    printf("&p   = %p    p   = %p    *p   = %d\n", (void *)&p, (void *)p, *p);
    printf("&pp  = %p    pp  = %p    *pp  = %p    **pp = %d\n",
           (void *)&pp, (void *)pp, (void *)*pp, **pp);

    **pp = 100;          /* 改的是 a */
    printf("经 **pp 修改后 a = %d\n", a);

    *pp = NULL;          /* 改的是 p 本身 */
    printf("经 *pp 修改后 p = %p\n", (void *)p);
    return 0;
}
```

3.6 指针与 const

写法	谁不能改	记忆口诀
const int *p;	不能通过 p 改指向的值; p 自己可以改指向	const 在 * 左边: 值不可改
int const *p;	同上 (写法等价)	同上
int * const p;	p 自己的指向不可改; 可以通过 p 改值	const 在 * 右边: 指针不可改
const int * const p;	两者都不可改	两边都有: 都不可改

表 3-2 const 与指针的四种组合

逆向里为什么常见 const char *

反编译代码里 (const char *)a1 通常表示"把 a1 当作只读字符串的首地址"。它不改变任何内存, 只声明"这里开始是一串字符"。看到这个转换, 基本可以判断后续会把它传给 strlen/strcmp/printf。

3.7 函数指针

函数也在内存里有地址, 因此"指向函数的指针"存在。逆向中函数指针极其常见: 虚表 (vtable)、回调、消息处理表都是它。

```

exp10.c —— 函数指针
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

int main(void)
{
    int (*fp)(int, int) = add;    /* fp 是指向"接受两个 int 返回 int"的函数 */

    printf("add 的地址 = %p\n", (void *)add);
    printf("fp 的值 = %p\n", (void *)fp);
    printf("fp(3,4) = %d\n", fp(3, 4));    /* 7 */
    printf("(*fp)(3,4) = %d\n", (*fp)(3, 4)); /* 7 —— 两种写法等价 */

    fp = sub;
    printf("fp(3,4) = %d\n", fp(3, 4));    /* -1 */
    return 0;
}

```

函数指针数组：模拟虚表

```

exp11.c —— 函数指针数组即虚表原型
#include <stdio.h>

int on_open (void) { puts("open"); return 0; }
int on_close(void) { puts("close"); return 0; }
int on_read (void) { puts("read"); return 0; }

/* 一张函数指针表，就是最朴素的虚表 */
int (*g_vtable[])(void) = { on_open, on_close, on_read };

int main(void)
{
    for (int i = 0; i < 3; i++)
        g_vtable[i]();    /* 依次调用 open / close / read */

    /* 从表里取出第 1 项并调用 */
    int (*fn)(void) = g_vtable[1];
    fn();
    return 0;
}

```

逆向里如何识别函数指针

在 x64 上，如果看到 `call qword ptr [rax+18h]`

这种"调用某个内存里的值"，那就是在通过函数指针调用。`+18h` 就是虚表中的第 3 项（每项 8 字节：0,8,10,18）。把这张表在 IDA 里还原成结构体，是逆向 C++ 程序的关键一步。

3.8 指针与字符串

字符串就是"以 0 结尾的字符数组"

```
char s1[] = "hello"; /* 6 字节的数组 (含结尾 \0) , 可修改 */
char *s2 = "hello"; /* 指向只读常量区的字符串字面量, 不可修改 */
char *s3 = s1; /* 指向数组首元素 */

printf("%s\n", s1); /* hello */
printf("%c\n", s1[1]); /* e */
printf("%c\n", *(s1 + 1)); /* e —— 同上 */
printf("长度 %zu\n", strlen(s1)); /* 5, 不含结尾 \0 */
```

为什么反编译代码里满屏是 (const char *)

字符串在内存里没有长度信息, 只有结尾的

0x00。所以要处理字符串, 只需要一个首地址, 编译器就把它标成 const char *。看到 (const char *) (a1 + 4112), 就应理解成: "从 a1+4112 开始, 有一串以 0 结尾的字符"。

第 4 章 结构体、对齐与填充

这一章解决逆向中最实际的问题：给定一串偏移量，还原出结构体定义。要做到这一点，必须理解对齐、padding（填充）和内存空洞。

4.1 为什么需要对齐

CPU 访问内存不是"任意字节都能随便取"的。多数架构上，访问一个 4 字节整数时，如果它的起始地址是 4 的倍数，硬件一次就能读完；否则可能要拆成两次访问再拼接，甚至直接触发异常。

- 对齐值（alignment）：每个类型有一个"对齐值"，通常等于它自己的大小（但不超过平台的默认最大对齐）。
- 对齐规则：结构体成员必须放在"对齐值的整数倍"偏移上，编译器为此插入无用字节。
- 填充（padding）：为对齐而插入的空白字节，里面是垃圾值，读它没有意义。

类型	大小	对齐值	地址必须是
char	1	1	任意地址
short	2	2	2 的倍数
int	4	4	4 的倍数
float	4	4	4 的倍数
long long	8	8	8 的倍数
double	8	8	8 的倍数
指针 (x64)	8	8	8 的倍数

表 4-1 常见类型的对齐值（x64 默认）

4.2 一个必须自己推一遍的结构体

sizeof(struct A) 是多少？先自己算，再看下面

```
struct A {
    char c;    /* 1 字节 */
    int i;     /* 4 字节 */
    short s;   /* 2 字节 */
    double d;  /* 8 字节 */
};
```

逐步推导（假设结构体本身按 8 字节对齐）：

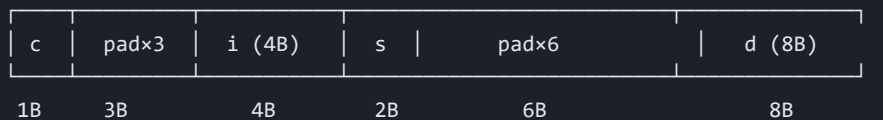
成员	大小	对齐	当前偏移	放得下吗	处理
c	1	1	0	可以	占 [0,1)
i	4	4	1	不行(1%4!=0)	填充 3 字节, 偏移升到 4 占 [4,8)
s	2	2	8	可以	占 [8,10)
d	8	8	10	不行(10%8!=0)	填充 6 字节, 偏移升到 16 占 [16,24)

结束偏移 24, 已是 8 的倍数, 无需再补尾部填充
 sizeof(struct A) = 24 (实际用到 1+4+2+8 = 15 字节, 浪费 9 字节)

图 4-1 struct A 的内存布局

内存布局 (每个格子 1 字节):

偏移 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 ... 23



= 24 字节, 其中 9 字节是填充 (永远不要读它们)

尾部填充也要算

结构体总大小必须补齐到"最大成员对齐值"的整数倍, 这样在数组里每个元素的起始地址才仍然满足对齐要求。所以 sizeof 往往是 8 的倍数。

4.3 调整成员顺序就能省内存

同样的成员, 换个顺序差 8 字节

```

struct Bad {
    char c;
    int i;
    short s;
    double d;
};

struct Good {
    double d;
    int i;
    short s;
    char c;
};

```

sizeof(struct Bad) = 24 sizeof(struct Good) = 16

Bad 的布局:	Good 的布局:
0 c	0 d (8B)
1 pad pad pad	8 i (4B)
4 i i i i	12 s (2B)
8 s s	14 c (1B)
10 pad×6	15 pad (1B) <-- 尾部对齐
16 d d d d d d d d	
	合计 16 字节, 省下 8 字节
合计 24 字节	

这条规律对逆向很有用

当你在逆向里发现一个结构体的大小时而 24 时而

32，不一定是搞错了，很可能是对方的声明顺序不同，或者用了不同的编译选项。判断依据永远是"你实际观察到的偏移"，而不是"你觉得合理的顺序"。

4.4 实验：打印真实偏移与大小

不要相信推演，直接让编译器告诉你。offsetof 是标准宏，必须掌握。

exp12.c —— offsetof 是你最好的朋友

```
#include <stdio.h>
#include <stddef.h>          /* offsetof */

typedef struct _MyData {
    int    id;                /* +0 */
    char   name[16];          /* +4 */
    void *next;               /* +20 -> 对齐后 +24 (x64) */
    double score;             /* +32 */
} MyData;

int main(void)
{
    printf("sizeof(MyData)   = %zu\n", sizeof(MyData));
    printf("offsetof(id)     = %zu\n", offsetof(MyData, id));
    printf("offsetof(name)   = %zu\n", offsetof(MyData, name));
    printf("offsetof(next)   = %zu\n", offsetof(MyData, next));
    printf("offsetof(score)  = %zu\n", offsetof(MyData, score));

    /* 用指针运算验证偏移 */
    MyData d;
    printf("&d=%p  (char*)&d.id    - (char*)&d = %lld\n",
           (void *)&d, (long long)((char *)&d.id - (char *)&d));
    printf("          (char*)&d.next  - (char *)&d = %lld\n",
           (long long)((char *)&d.next - (char *)&d));
    return 0;
}
```

逆向时的对照方法

反编译代码给了你一堆偏移 (+0、+4、+0x18、+0x20)。在本地写一个候选结构体，用 offsetof 打印各个成员的偏移，两个列表一对，就能验证你的还原是否正确。这是最可靠的还原手段。

4.5 嵌套结构体与数组作为成员

嵌套成员的偏移 = 外层偏移 + 内层偏移

```
typedef struct _Vec3 {
    float x, y, z;      /* +0 +4 +8, 大小 12 */
} Vec3;

typedef struct _Entity {
    int id;              /* +0 */
    Vec3 pos;            /* +4 嵌套: 占用 12 字节, 内部偏移 0/4/8 */
    Vec3 vel;            /* +16 再嵌一个 */
    char name[32];       /* +28 */
    int flags;           /* +60 */
} Entity;               /* 大小 64 (对齐 4) */

/* 访问嵌套成员的偏移是"相加"的: */
/* e.pos.y -> &e + 4 + 4 = &e + 8 */
/* e.vel.z -> &e + 16 + 8 = &e + 24 */
/* e.name[i] -> &e + 28 + i */
/* e.flags -> &e + 60 */
```

4.5.1 实验 13: 验证嵌套偏移

exp13.c —— 嵌套偏移相加

```
#include <stdio.h>
#include <stddef.h>

typedef struct _Vec3 { float x, y, z; } Vec3;

typedef struct _Entity {
    int id;
    Vec3 pos;
    Vec3 vel;
    char name[32];
    int flags;
} Entity;

int main(void)
{
    printf("sizeof(Vec3)      = %zu\n", sizeof(Vec3));
    printf("sizeof(Entity)    = %zu\n", sizeof(Entity));
    printf("offsetof(pos)          = %zu\n", offsetof(Entity, pos));
    printf("offsetof(vel)           = %zu\n", offsetof(Entity, vel));
    printf("offsetof(name)          = %zu\n", offsetof(Entity, name));
    printf("offsetof(flags)         = %zu\n", offsetof(Entity, flags));

    /* 嵌套成员: pos.y 的偏移 = offsetof(pos) + offsetof(Vec3, y) */
    printf("pos.y 的偏移    = %zu (应为 %zu + 4)\n",
        offsetof(Entity, pos) + offsetof(Vec3, y),
        offsetof(Entity, pos));
    return 0;
}
```

4.6 位域与紧凑结构体 (了解即可)

位域与 #pragma pack

```
/* 位域：按位分配，用于协议头、标志位 */
typedef struct _Flags {
    unsigned int a : 1; /* 占 1 位 */
    unsigned int b : 3; /* 占 3 位 */
    unsigned int c : 4; /* 占 4 位 */
} Flags; /* 共 8 位 = 1 字节（同一存储单元内） */

/* 紧凑结构体：禁止填充，用于磁盘/网络格式 */
#pragma pack(push, 1)
typedef struct _Packed {
    char    tag; /* +0 */
    int     value; /* +1 未对齐！（正常应为 +4） */
    short   crc; /* +5 */
} Packed; /* 大小 = 7，没有填充 */
#pragma pack(pop)
```

逆向中看到"奇怪的偏移"先怀疑 pack

如果观察到 int 成员出现在偏移 1、5 这种非 4 倍数位置，几乎可以确定该结构体使用了 pack(1)（协议头、文件格式、游戏存档常见）。在 IDA 里可以给结构体设置"packed"属性来匹配。

第 5 章 逆向代码逐层拆解

现在把前面所有知识合起来，彻底讲清讲义开头那两行代码。这一章会先建立“偏移表 = 结构体”的对应关系，再逐层剥开表达式。

5.1 目标代码与它的含义

目标：两行必须能自己讲明白的代码

```
*( _BYTE *) ( *( _QWORD *) ( a1 + 8208 ) + 4120LL )
( const char *) ( a1 + 4112 )
```

先给结论，再给推导。这两行说明：a1 指向一个很大的对象（至少 12328 字节）；在 a1+8208 处存放着一个指针（8 字节，所以是_QWORD）；该指针指向另一个对象；在那个对象里偏移 4120 处有一个字节；而 a1+4112 处开始是一串字符（字符串）。

一个非常关键的观察

这些偏移（4112、8208）都是 16

的倍数，而且彼此相距很远。这几乎不可能是“手写的小结构体”，而更像是“若干个同类型子结构体顺序排布”或者“数组”。 $8208 - 4112 = 4096 =$

$0x1000$ ，这个数字太规整了，暗示有一大片定长区域。

5.2 第一步：把数字都换成十六进制

```
4112 = 0x1010
4120 = 0x1018      (4112 + 8)
8208 = 0x2010
```

重写目标代码：

```
*( _BYTE *) ( *( _QWORD *) ( a1 + 0x2010 ) + 0x1018LL )
( const char *) ( a1 + 0x1010 )
```

现在结构立刻清楚了很多：

0x1010 和 0x1018 只差 8 字节 —— 很可能是同一结构的相邻成员

0x2010 是 0x1010 的两倍 —— 很像“数组第 2 个元素”

0x1000 是常见页大小（4096），偏移 0x1010 像是“头 + 数据”的布局

为什么一定要换成十六进制

逆向里的偏移几乎都来自结构体布局，而结构体布局由“对齐”决定，对齐值都是 2

的幂。所以偏移在十六进制下呈现出极规律的形状（0x1010、0x1018、0x2010、0x2020...），在十进制下则毫无规律。换算成十六进制再找规律，是逆向的基本功。

5.3 第二步：先猜出偏移表，再反推结构体

逆向结构体的通用套路是：把代码里出现的所有偏移收集起来，按"谁和谁只差一个类型大小"分组，然后猜测成员类型。

观察到的大致偏移	推断的类型	理由
+0x1010	char[?] 或 指向字符串的字段	被转成 const char *, 是字符串起点
+0x1018	位于"另一个对象"内部, 偏移 0x1018	先解引用 _QWORD, 再加 0x1018
+0x2010	_QWORD (8 字节指针)	被 _QWORD * 解引用, 说明这里存指针
+0x1000 附近	很可能是另一个子结构或数组	0x1010 - 0x1000 = 0x10

表 5-1 从偏移反推类型

5.3.1 为什么 0x2010 处是 _QWORD 而不是 _DWORD

这是最关键的一步推断。代码写的是 `*(_QWORD *)(a1 + 0x2010)`, 意思是"从 `a1+0x2010` 开始读 8 个字节, 把这 8 个字节当成一个地址"。如果那里其实是 4 字节的 `_DWORD`, 编译结果不会是这个形状。所以可以断定:

- 推论 1: 这是一个 64 位程序 (否则指针不会是 8 字节)。
- 推论 2: `a1+0x2010` 处存放的是一个指针成员, 而不是整数。

逆向中"宽度即类型"的推理原则

`_BYTE` / `_WORD` / `_DWORD` / `_QWORD` 直接对应 1/2/4/8 字节。看到 `_QWORD` 就往"指针或 64 位整数"想, 看到 `_DWORD` 就往"int 或 float"想。这是从反编译代码里提取类型信息最快、最可靠的手段。

5.4 第三步：逐层拆解第一个表达式

面对嵌套表达式, 从最里面往外剥, 一次只做一件事, 并且每一步都写清楚"现在的类型是什么、值是什么"。

从内到外的五层拆解

原文:

```
*(_BYTE *)(*(_QWORD *)(a1 + 8208) + 4120LL)
```

第 0 层: `a1`

类型: 一个指针 (参数), 指向某个大对象

值: 某个地址, 记作 `Base`

第 1 层: `a1 + 8208`

含义: `Base` 向前走 8208 个字节

类型: 仍是指针 (但语义上已不是"对象开头")

值: `Base + 0x2010`

第 2 层: `*(_QWORD *)(a1 + 8208)`

含义: 从 `Base+0x2010` 处读 8 个字节, 当成一个地址

类型: `_QWORD`, 即"在这里取出了一个指针"

值: 记作 `P1` (某个对象的首地址)

第 3 层: `*(_QWORD *)(a1 + 8208) + 4120LL`

含义: `P1` 再向前走 4120 个字节

注意: 这里加的是"1 字节 × 4120", 因为下一步要按 `_BYTE` 读, 所以编译器把它写成"字节偏移"的形式

值: `P1 + 0x1018`

第 4 层: `*(_BYTE *)(...)`

含义: 从 `P1+0x1018` 处读 1 个字节

类型: `unsigned char`

值: 某个字节, 记作 `B`

所以整个表达式的结果 = 位于 `P1+0x1018` 的那 1 个字节。

一个细节值得注意

第 3 层里 4120 后面跟着 `LL` 后缀 (`long long`)。这说明编译器把偏移运算放在了 64 位宽度上, 进一步印证是 64 位程序, 同时也提示"这个加法的量纲是字节" (因为紧接着是 `_BYTE` 解引用)。

5.5 第四步: 拆解第二个表达式

两层的拆解

原文：

```
(const char *) (a1 + 4112)
```

第 0 层: a1

值: Base

第 1 层: a1 + 4112

值: Base + 0x1010

含义: 从 Base 偏移 0x1010 处开始

第 2 层: (const char *)

含义: 把这个地址解释为"字符串起始地址"

不改内存, 只改变"如何解读"

所以整个表达式的结果 = 指向字符串 "..." 的首地址,
该字符串位于 Base + 0x1010 处, 以 0x00 结尾。

把两个结果放在一起看, 规律浮现:

Base + 0x1010 -> 字符串起点 (const char *)

Base + 0x2010 -> 一个指针 (_QWORD)

$0x2010 - 0x1010 = 0x1000 = 4096$

猜法一: 这是两个并列的大数组/缓冲区, 每个 0x1000 字节。

0x1010 说明"前 0x10 字节是头部, 之后是 0x1000 字节的数据"。

猜法二: Base 是一个"表", 0x1010 处是表项 0 的某个字段,

0x2010 处是表项 1 的同一字段 (表项大小 0x1000)。

5.6 第五步: 还原成结构体定义

把上面的推断写成 C

代码。注意: 逆向得到的结构体是"能解释所有偏移"的最简结构, 不一定是原作者写的样子, 这完全没问题。

还原后的结构体与等价写法

```
/* 被 a1 + 0x2010 指向的那个对象（偏移 0x1018 处有一个字节） */
typedef struct _SubObject {
    unsigned char unknown_0000[0x1018]; /* 我们不关心的部分 */
    unsigned char flag_or_state; /* +0x1018 <- 目标字节 */
    /* ... 后面还能继续填 */
} SubObject;

/* a1 指向的对象 */
typedef struct _MainObject {
    unsigned char unknown_0000[0x1010]; /* 前 0x1010 字节：未分析 */
    char name[0x1000]; /* +0x1010 字符串区（以\0 结尾） */
    SubObject *sub; /* +0x2010 指向 SubObject 的指针 */
    /* ... */
} MainObject;

/* 于是原来的代码可以写成人类可读的形式： */
unsigned char v = a1->sub->flag_or_state;
const char *s = a1->name;
```

这个过程才是逆向结构体的标准流程

不要一上来就试图猜出完整结构体。正确顺序是：① 收集代码中出现的全部偏移；② 换成十六进制找规律；③ 用"宽度"判断类型（_BYTE=1、_WORD=2、_DWORD=4、_QWORD=8）；④ 把未知区域先声明成 unknown_NNNN 字节数组占位；⑤ 写成结构体后用 offsetof 验证偏移是否完全吻合；⑥ 随着分析更多函数，逐步替换掉占位数组。

5.7 从汇编角度再确认一次

反编译代码是编译器"优化后"的结果。看汇编能确认宽度与层级理解是否正确。

关键指令解读

```
; 目标:  *(_BYTE *)(*(_QWORD *)(a1 + 8208) + 4120LL)
; 假设 a1 在 rcx, 64 位 Windows 调用约定

mov    rax, [rcx+2010h]    ; 从 a1+0x2010 取一个 8 字节指针 -> rax
movzx  eax, byte ptr [rax+1018h] ; 从 rax+0x1018 取 1 字节，零扩展到 32 位
                                ; 结果就在 eax 里，作为函数返回值

retn

; 目标:  (const char *)(a1 + 4112)
lea    rax, [rcx+1010h]    ; 只是取地址，不访问内存

retn
```

指令片段	含义	对应 C 代码
mov rax, [rcx+2010h]	从内存读 8 字节到 rax (rcx 是基址)	rax = *(_QWORD *)(a1 + 0x2010)
movzx eax, byte ptr [rax+1018h]	从内存读 1 字节并零扩展	*(_BYTE *)(rax + 0x1018)
lea rax, [rcx+1010h]	只做地址加法，不读内存	(const char *)(a1 + 0x1010)

表 5-2 三条指令与 C 代码的对应

mov 和 lea 的区别是分水岭

mov 会真的读写内存；lea 只算地址。看到 lea

就知道后续要拿这个地址去用（传参、存指针、做字符串），而不是取里面的内容。这一条能帮你快速区分“取指针”和“取指针指向的值”。

5.8 完整可运行例子：自己造一个同样的结构体

最好的验证方式是自己写一份代码，让它编译出形状类似的访问。

exp14.c —— 用真实结构体重放目标代码

```
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <stddef.h>

#define NAME_AREA    0x1000

typedef struct _SubObject {
    uint8_t  pad[0x1018];
    uint8_t  flag_or_state;          /* +0x1018 */
} SubObject;

typedef struct _MainObject {
    uint8_t  pad[0x10];              /* 前 0x10 字节 */
    char     name[NAME_AREA];        /* +0x10 ... 到 +0x1010 */
    SubObject *sub;                  /* +0x1010 + 0x1000 = +0x2010 */
} MainObject;

int main(void)
{
    printf("offsetof(name) = 0x%zX\n", offsetof(MainObject, name));
    printf("offsetof(sub)  = 0x%zX\n", offsetof(MainObject, sub));

    static SubObject sub;
    static MainObject obj;

    sub.flag_or_state = 0x5A;
    obj.sub = &sub;
    strcpy(obj.name, "reverse engineering");

    /* 这两行就是"目标代码"的人类可读版本 */
    uint8_t    v = *(uint8_t *)*((uint64_t *)((char *)&obj + 0x2010) + 0x1018);
    const char *s = (const char *)((char *)&obj + 0x1010);

    printf("v = 0x%02X\n", v);
    printf("s = %s\n", s);
    printf("obj.sub->flag_or_state = 0x%02X\n", obj.sub->flag_or_state);
    return 0;
}
```

为什么这里的偏移和讲义开头略有不同

讲义开头是 $a1 + 4112$ (0x1010) 取字符串、 $a1 +$

8208 (0x2010) 取指针。本例子为了用数组表达"前 0x10 字节 + 0x1000 字节名字区", 把 name 放在 0x10 处、结束于 0x1010, sub 落在

0x2010。这说明: 还原结构体时, "成员到底叫什么"并不重要, "偏移能对上、类型宽度能解释读写"才是正确性的唯一标准。

第 6 章 自测：可能卡住你的地方

这一章把所有容易产生误解的点集中列出。建议先只看问题、动手想，再翻到后面的答案。

6.1 二十个高频困惑点

1. 为什么指针还要分类型？地址不就是一个数字吗？
2. `int *p;` 里的 `*` 和 `*p = 1;` 里的 `*` 是同一个东西吗？
3. `p + 1` 和 `(char *)p + 1` 有什么区别？
4. 为什么 `sizeof(数组)` 和 `sizeof(数组作参数的形参)` 不一样？
5. `a[i]` 和 `*(a+i)` 完全等价，那 `2[a]` 为什么也合法？
6. 为什么 `int a[5];` 之后 `a` 不能赋值（不能用 `a = ...`）？
7. `&a` 和 `&a[0]` 的值一样，类型一样吗？
8. 结构体成员之间的“空洞”里到底是什么？能读吗？
9. 为什么结构体大小不等于成员大小之和？
10. 为什么把成员顺序换一下，结构体就变小了？
11. 为什么 x64 下指针是 8 字节，但 `long` 在 Windows 上还是 4 字节？
12. `*(QWORD *)p` 和 `*(DWORD *)p` 在读同一地址时，结果一样吗？
13. 为什么反编译代码里到处是强制类型转换？
14. 函数指针的 `(*fp)()` 和 `fp()` 为什么等价？
15. 字符串为什么需要结尾的 0？没有会怎样？
16. `const char *p` 里的 `const` 到底限制了谁？
17. 为什么观察到 `int` 成员出现在偏移 1？
18. `mov` 和 `lea` 的核心区别是什么？
19. 为什么逆向时要把十进制偏移换成十六进制？
20. 还原结构体时，成员名字重要还是偏移重要？

6.2 逐条解答

Q1 为什么指针还要分类型

因为类型决定了“读几个字节”和“+1 走多远”。`0x1000` 这个地址，用 `char*` 读得到 1 字节，用 `int*` 读得到 4 字节。同一个地址，不同类型的解释结果可以完全不同。

Q2 声明里的 * 与表达式里的 *

不是同一个东西。声明里的 `*` 属于类型的一部分（`int *` 是指针类型）；表达式里的 `*` 是一元运算符“解引用”。判断方法：看这个 `*` 前面是不是一个类型名。

Q3 p+1 与 (char*)p+1

如果 p 是 int*, p+1 走 4 字节, 而 (char*)p+1 走 1 字节。前者是"下一个 int", 后者是"下一个字节"。这就是为什么扫描内存要用 char*, 读取整数要用 int*。

Q4 sizeof 数组与形参

数组名在传参时会退化成指针, 长度信息丢失。所以 sizeof 在函数内得到的是指针大小。这就是为什么 C 函数必须额外传一个长度参数。

Q5 2[a] 为什么合法

因为 a[i] 定义上就是 *(a+i), 而加法满足交换律, *(a+i) 等于 *(i+a), 写成下标形式就是 i[a]。结论: 合法但不要这么写, 它只是帮助你理解下标本质。

Q6 数组名为什么不能赋值

数组名在大多数场合退化为"指向首元素的指针值", 但它本身不是一块可写的变量, 它是数组这个对象的标识。所以 a = p; 编译不过, 而 char *q = a; 可以。

Q7 &a 与 &a[0]

数值相同 (都是数组首地址), 但类型不同: &a[0] 是 int*, &a 是 int(*)[5] (指向整个数组的指针)。因此 &a + 1 会跳 20 字节 (整个数组), 而 &a[0] + 1 只跳 4 字节。这是指针算术里最容易算错的地方之一。

Q8 空洞里是什么

是未初始化的垃圾字节, 内容不确定, 可能残留上一次的栈或堆数据。读它在 C 里不是未定义行为 (对 unsigned char 而言), 但它的值没有任何语义。逆向时不要试图解释 padding 的内容。

Q9 为什么大小不等于成员之和

因为对齐。每个成员必须放在对齐值的整数倍偏移上, 编译器会插入填充字节; 结构体末尾还要补齐到最大对齐值的倍数。

Q10 换个顺序为什么变小

把大成员放前面、小成员凑在一起, 可以减少因对齐产生的空洞。经验法则: 按对齐值从大到小排列成员。

Q11 为什么 Windows 上 long 还是 4 字节

因为 Windows 使用 LLP64 模型 (long long 和指针是 64 位, long 仍是 32 位), 而 Linux 使用 LP64 (long 也是 64 位)。这直接影响结构体布局, 是跨平台逆向必须确认的事。

Q12 _QWORD 与 _DWORD 读同一地址

不一样。_QWORD 读 8 字节, _DWORD 读 4 字节。如果那个位置实际存的是 8 字节指针, 用

_DWORD 读只会得到低 4 字节，在 x64 上会得到一个错误的（被截断的）地址。所以宽度判断错误会直接导致分析失败。

Q13 为什么到处是强转

为了改变"读取宽度、解释方式和步进倍率"。编译器在优化后会用统一的指针类型做运算，需要读整数就转成 _DWORD*，需要读字节就转成 _BYTE*，需要按字节扫描就转成 char*。

Q14 (*fp)() 与 fp()

两者完全等价。函数调用运算符 () 的左边需要的是一个函数指针，fp 本身就是，(*fp) 解引用后仍是同一个地址。编译器接受两种写法只是为了可读性（提醒读者 fp 是指针）。

Q15 字符串为什么需要结尾 0

因为 C 字符串不保存长度，所有字符串函数都靠扫描到 0x00 来判断结束。如果没有 0，strlen 会一直往后读，直到撞上内存里的某个 0，轻则结果错误，重则越界崩溃。这是最常见的漏洞来源之一。

Q16 const char *p 限制了谁

限制的是"通过 p 修改它指向的内容"，不限制 p 自身改变指向。要限制 p 自身，需要写成 char * const p。

Q17 为什么 int 成员在偏移 1

几乎可以断定该结构体用了 #pragma pack(1) 或 __attribute__((packed))。常见于网络协议头、文件格式、游戏存档。在 IDA 中可以给结构体设置 packed 属性以匹配。

Q18 mov 与 lea

mov 访问内存（读或写数据），lea 只计算地址、不访问内存。在逆向里，lea 常出现在"准备把地址传给函数"或"取字符串指针"的位置。

Q19 为什么换成十六进制

因为结构体偏移由 2 的幂次对齐决定，在十六进制下规律非常明显（0x10、0x18、0x20、0x1010、0x2010 之间的倍数关系一眼可见），十进制下则完全看不出来。

Q20 成员名字还是偏移

偏移和宽度决定正确性，名字只是可读性。一个结构体只要有正确的偏移与类型，哪怕成员全叫 field_0、field_4，也能正确解释程序行为。随着理解加深再改名即可。

6.3 动手练习

下面五题请先自己算或自己写代码，再看答案。

练习 1

在 64 位 Windows 上，sizeof(struct S) 是多少？各成员偏移是多少？

```
struct S {  
    char a;  
    int b;  
    char c;  
    long long d;  
};
```

练习 2

下面代码输出什么？（先不要编译）

```
int a[3] = { 1, 2, 3 };  
int *p = a + 1;  
printf("%d %d\n", *p, p[-1]);
```

练习 3

已知 a1 是结构体指针，写出等价的人类可读写法：

```
*(_DWORD *)(a1 + 16) = 100;  
*(_BYTE *)(*(_QWORD *)(a1 + 32) + 8) = 1;
```

练习 4

把上面的结构体还原出来：要求写出 typedef 定义，并说明每个成员的类型与偏移。

练习 5

为什么下面这段代码在 64 位程序里是危险（且很可能错误）的？

```
int *p = (int *)SomePointer;  
int v = *(int *)((char *)p + 1);
```

6.4 练习答案

答案 1

成员	大小	对齐	偏移推演	结果偏移
a	1	1	起点 0	0
b	4	4	1 不是 4 的倍数 -> 补到 4	4
c	1	1	8	8
d	8	8	9 不是 8 的倍数 -> 补到 16	16

结束偏移 24, 最大对齐 8, 24 已是 8 的倍数
sizeof(struct S) = 24 (实际只用 14 字节, 填充 10 字节)

答案 2

输出 "2 1"。p 指向 a[1], 所以 *p 是 2; p[-1] 等价于 *(p-1), 即 a[0], 是 1。负下标是合法且常用的写法。

答案 3

```
/* 原始 */
*(DWORD*)(a1 + 16) = 100;
*(BYTE*)(*(QWORD*)(a1 + 32) + 8) = 1;

/* 人类可读版本 (a1 是 SomeStruct *) */
a1->field_16 = 100; /* 某个 4 字节成员位于 +0x10 */
a1->field_32->field_8 = 1; /* +0x20 处有一个指针, 指向的对象 +0x8 处是 1 字节 */
```

答案 4

```
typedef struct _Inner {
    unsigned char unknown_0000[8]; /* 前 8 字节未分析 */
    unsigned char flag; /* +0x8 */
    /* ... */
} Inner;

typedef struct _Outer {
    unsigned char unknown_0000[0x10]; /* +0x0 ~ +0xF 未分析 */
    int field_16; /* +0x10 被写入 100, 是 4 字节 */
    unsigned char unknown_0014[0xC]; /* +0x14 ~ +0x1F (为满足 8 字节对齐保留) */
    Inner *field_32; /* +0x20 8 字节指针 (x64) */
} Outer;

/* 验证: offsetof(Outer, field_16) == 0x10
    offsetof(Outer, field_32) == 0x20
    offsetof(Inner, flag) == 0x8 */
```

注意 field_32 是 8 字节指针, 它的偏移必须是 8 的倍数, 所以 0x10 + 4 = 0x14 之后需要补到 0x18 或直接放到 0x20; 具体是 0x18 还是 0x20

取决于之前还有没有其他成员。这正说明: 只有结合更多代码才能确定, 不能凭空猜。

答案 5

`(char *)p + 1` 只前进 1 字节，得到的地址通常不是 4 的倍数，未对齐读取 `int`。在 x86 上可能“能跑但变慢”，在 ARM 等架构上会直接抛异常；而且从语义上讲，跨过 1 字节去读一个 `int`，读到的是“上一个 `int` 的后 3 字节 + 下一个 `int` 的第 1 字节”的拼接，几乎肯定不是想要的结果。

这一题的意义

它同时考了指针算术的量纲、对齐要求和“跨类型读取”三个知识点。能把这一题讲清楚，说明前面的内容基本吃透了。

6.5 学习路线建议

- 1. 把第 0 章的三个实验亲手跑一遍，确认输出与自己预期一致。
- 2. 把第 3 章的 `exp7` 改一改：把 `int` 换成 `double`、`short`，观察 `+1` 的距离变化。
- 3. 把第 4 章的 `exp12` 改成你自己设计的一个结构体，用 `offsetof` 验证你的推演。
- 4. 找一个真实的小程序（自己写一个带结构体的），用调试器查看结构体在内存里的样子。
- 5. 用 IDA Free 或 Ghidra 打开它，把反编译出的偏移与你的结构体对照。
- 6. 回到第 5 章，把目标两行代码的结构体定义换成你自己的想法，看能否解释通。
- 7. 最后找一道 CTF 逆向题（简单的 `crackme`），尝试还原其中一个结构体。

6.6 术语对照表

中文	英文	说明
解引用	dereference	通过指针访问它指向的内存
取地址	address-of	用 <code>&</code> 得到一个变量的地址
字节序	endianness	多字节数据的字节排列顺序
小端	little-endian	低位字节在低地址
大端	big-endian	高位字节在低地址
对齐	alignment	数据地址必须是某个值的整数倍
填充	padding	为对齐而插入的无用字节
空洞	hole / slack	填充造成的未使用空间
退化	decay	数组名在表达式中变成指针
虚表	vtable	虚函数的函数指针数组
偏移	offset	相对基址的字节距离
强制类型转换	cast	改变编译器对某地址的解释方式
紧凑结构体	packed struct	禁用填充的结构体
位域	bit-field	按位分配的成员
类型双关	type punning	用不同类型读写同一块内存

表 6-1 常用术语中英对照

6.7 一句话总结

把这一章记住一句话就够了

指针就是"一个数字 +

一个类型": 数字是地址, 类型决定读多宽、走多远。结构体就是"若干个成员按对齐规则排开", 偏移由前面成员的大小与对齐决定。逆向的全部工作, 就是把这套规则反过来用: 从代码里的偏移和宽度, 倒推出成员的类型和顺序。